mb-free___transport-rendezvous-8192

An AgentMPI run analysis

Abstract

This is the first cell of a ten-cell transport sweep at which the mode is not a preference. Eight agent-free ranks ring-exchange an 8,192-word payload under `Mode.RENDEZVOUS`: 34 events, 0.06 s, all eight messages delivered, zero admissions, a context high water of zero on every rank and 86,232 tokens deferred. Its eager twin, given the same payload, produced 18 events and no messages at all — `results/microbench/free-transport.json` records `completed: false`, `n_failed: 8`, `ERR_CONTEXT_OVERFLOW` — because the payload's 10,779 estimated tokens exceed the destination's 4,096-token unexpected-message credit by 2.63× and `_await_eager_credit` refuses it before writing a fabric row. The pair is unusually clean evidence: the two runs put the byte-identical blob `3fbdeea17da4` into their content stores, address the same ring, and differ in nothing except what the envelope carries. Eager put the payload in it and was refused; rendezvous put a handle in it and was not. This is the cell that turns AgentMPI's transport-mode argument from a design preference into a requirement.

1 What this run is

property	value
run	`mb-free__transport-rendezvous-8192`
experiment	-
campaign label	-
declared world size	8
ranks appearing in the log	8
executors	`none` × 8
events	34
wall time	0.06 s
job outcome	completed

2 Headline measurements

metric	value	meaning
achieved parallelism	0.00×	total busy time over wall time
parallel efficiency	0.0%	achieved parallelism per rank
idle fraction	100.0%	rank-seconds paid for and unused
peak ranks busy	0	of 8 in the world
serial fraction of active time	0.0%	time with exactly one rank busy
load imbalance	0.00×	slowest rank's busy time over the mean
coordination share	0.0%	rank-seconds blocked in collectives, of those available
coordination in flight	0.0%	wall clock with any rank inside a collective
rank reattachments	0	fresh agent processes that took over a rank role
agent calls	0	invocations that reached a model
agent latency p50 / p95	0.00 / 0.00 s	per invocation
messages	8	0 eager, 8 rendezvous
tokens sent	86,232	86,232 deferred under rendezvous
tokens in / out	0 / 0	prompt and completion
cost	$0.0000	0.0000 $/kTok out

3 What the analysis notices

Nothing anomalous was detected mechanically.

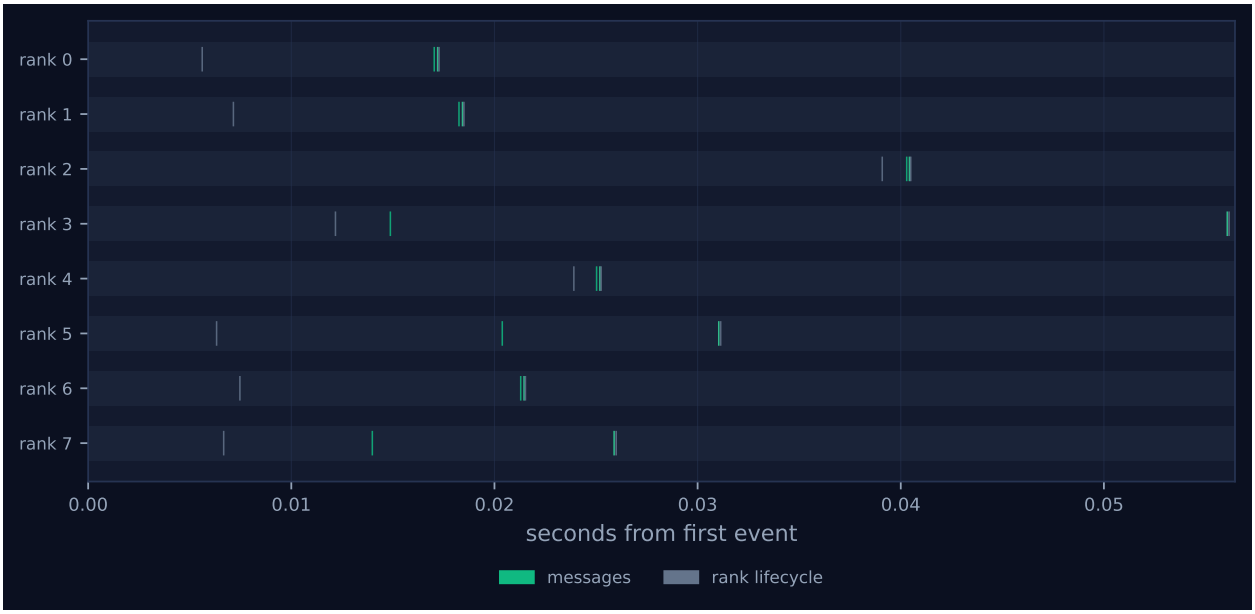

Figure 1: Execution timeline, one lane per rank. Bars are intervals when a rank held a task; ticks are messages; diamonds are window operations, lifecycle transitions, failures, and recovery actions. Drawn in the trace viewer's palette so the same shapes read the same way in both.

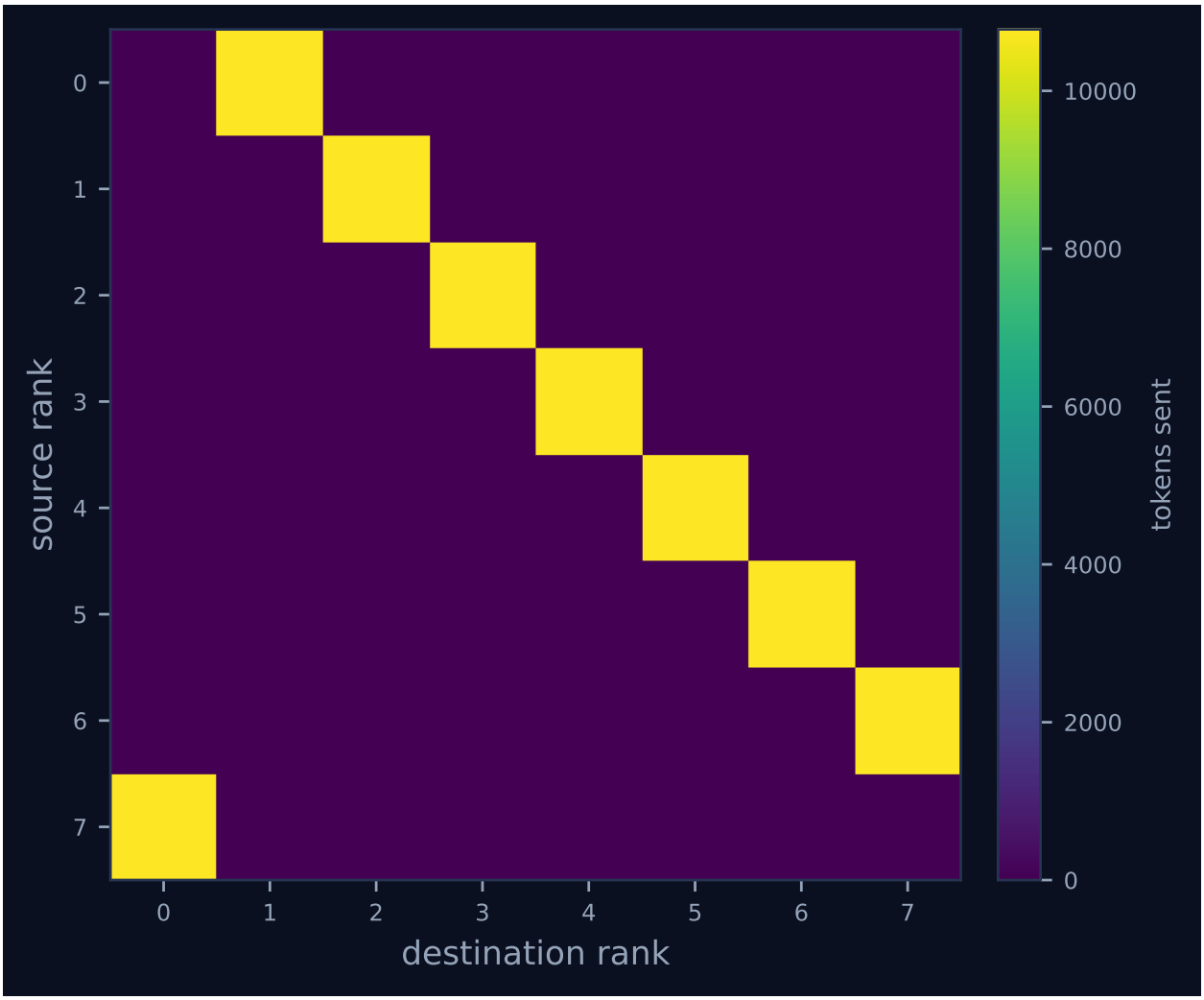

Figure 2: Communication matrix: tokens sent from each rank to each rank. The algorithm's shape is visible here — a tree concentrates traffic on a root, a ring forms a diagonal band, and an unintended all-to-all fills the plane.

4 Per-rank detail

rank	busy (s)	occ. (%)	tasks	calls	sent	recv	tok. sent	ctx (%)	state
0	0.00	0.0	0	0	1	1	10,779	0.0	finalized
1	0.00	0.0	0	0	1	1	10,779	0.0	finalized
2	0.00	0.0	0	0	1	1	10,779	0.0	finalized
3	0.00	0.0	0	0	1	1	10,779	0.0	finalized
4	0.00	0.0	0	0	1	1	10,779	0.0	finalized
5	0.00	0.0	0	0	1	1	10,779	0.0	finalized
6	0.00	0.0	0	0	1	1	10,779	0.0	finalized
7	0.00	0.0	0	0	1	1	10,779	0.0	finalized

Per-rank behaviour. Occupancy is busy time over the rank's own lifetime, so a rank that joined late is not penalised for the time before it existed.

5 Collectives, against the cost model

This run invoked no collectives.

6 Reading the timeline

Figure 1 shows eight lanes carrying two grey lifecycle diamonds and two green message ticks each, and no bars: the executor is `none` on every rank, there were zero agent calls, and no `task.*` event exists for a bar to span. Read "idle fraction 100.0%" and "achieved parallelism 0.00×" in the headline table as saying that no rank ever held a task. Ranks arrive over a 33 ms window inside a 56 ms span, and the receive latencies have grown: median 10 ms and maximum 31 ms, against 5 ms and 7 ms at 2,048 words. That is not the received payload moving — nothing is transferred on a rendezvous receive, and `sent_at` is stamped after the sender's own `fabric.blobs.put` has already completed. It is that every rank sends before it receives, so a message's measured latency includes the receiver's own time writing its 40 KiB outgoing blob before it ever looks in its mailbox. The same term triples the span against the 2,048-word cell's 20 ms. Nothing blocked: there is no `transport.credit_stall` in the log, and none is possible, because the eager credit mechanism does not apply to rendezvous traffic at all. Figure 2 shows the ring as a single off-diagonal band of 10,779 tokens per edge.

The comparison worth making is with a figure that does not exist. The eager cell at this size has no `figures/comm.pdf` — `analyze_run.py` does not draw a matrix for a run with no traffic — and its timeline has only the grey lifecycle marks, with the viewer's legend omitting the messages swatch entirely. Here the rank health table reads 0% occupancy across all eight ranks and the stats row reads `TOKENS DEFERRED 86.2k`; there it reads 0% occupancy and `0.0k` deferred, and the zeros mean the opposite thing. This is the sweep's sharpest illustration of why the trace must be read alongside the results file: two runs whose dashboards both show `FAILURES 0` and all ranks `finalized`, one of which delivered everything and one of which delivered nothing.

7 Interpretation

By construction: the world size, the ring mapping, one send and one receive per rank, and the mode — and here the forcing runs the other way from the eager column. Every `rank.init` records `eager_limit: 1000000000`, so `_choose_mode`'s automatic threshold is off; had it been left at the shipped `DEFAULT_EAGER_LIMIT` of 2,048 tokens, the runtime would have selected rendezvous for this 10,779-token payload on its own. This cell is what AgentMPI does by default, and its eager twin is what happens when a harness overrides that default at a size where the override cannot work. That framing matters for reading the sweep: the failure in the eager column is not AgentMPI failing, it is AgentMPI refusing an instruction it was given, and refusing it early, locally and with the remedy named in the exception message.

The mechanism of the contrast is worth stating precisely, because the two runs are so nearly identical that it is easy to assume rendezvous saved something physical. It did not. Both runs' `send` calls reached `fabric.blobs.put(payload)` and wrote all 40,960 bytes; the blob is present on disk in *both* run directories, under the same digest, because the content is the same and the store is content-addressed. Both would have sent eight envelopes. The divergence happens at the next line: `_choose_mode` returns `Mode.EAGER` for the twin, which enters `_await_eager_credit`, which compares 10,779 against the destination's `unexpected_limit` of 4,096 and raises `AmpiContextOverflow` on the spot — before the `messages` row, before the sequence-number increment, which is why that run's fabric has empty `messages` and `seqs` tables. Here `_choose_mode` returns `Mode.RENDEZVOUS`, the credit check is skipped entirely, and the envelope goes out carrying digest, token count, kind and a one-line synopsis. The difference between a run that works and a run that does not is one branch on a mode flag. Note also which limit did the refusing: 10,779 tokens would have fitted inside the receiver's 32,768-token context budget, at 33% occupancy. The eager path did not fail because the message was too big for the receiver; it failed because it was too big for the receiver's unexpected-message pool, a flow-control bound set at $\frac{1}{8}$ of the context budget in this benchmark.

The deferral figure needs one correction. `metrics.json` reports 86,232, which is $8 \times 10{,}779$ — one rendezvous send per rank, exactly equal to `tokens sent` because all the traffic is rendezvous, and consistent with the 128-, 512- and 2,048-word cells at 4.00× the last of them for 4× the payload. The results file records 172,464, and the per-rank events show `tokens_deferred: 21558` against `tokens_sent: 10779`: twice what the rank transmitted. Before commit `0fea51d2`, `RankCost.tokens_deferred` accumulated on rendezvous sends *and* on every non-admitted receive, so each rank counted its own send and its predecessor's arrival. The commit split the field into `tokens_deferred` (send-side, rendezvous only) and `tokens_unadmitted` (receive-side, any mode), under which this run reports 86,232 of each. `cost.summarise` derives from the log and was always right, which is why the viewer and `metrics.json` already show the corrected number. The fix's summary holds that the paper's transport table is unaffected because the microbench admits its receives; that is true of the eager rows, which correctly read zero, and false of the rendezvous rows, since `bench_transport` never passes `admit` and rendezvous defaults it to `False`. Every rendezvous entry in that table is exactly doubled.

8 Threats to this reading

The strongest caveat is that this cell's advantage is entirely deferral, and deferral is not delivery. No `fetch` is called anywhere in the transport sweep, so the eight receivers finished holding a digest, a token count and a synopsis of a document they never read. If the workload genuinely needed the

content, a rank would have to call `fetch` with `admit=True` and would then charge 10,779 tokens to a 32,768-token budget — affordable once, not four times. The cell therefore shows that a large artefact can be *addressed* without cost, not that it can be *used* without cost, and the mechanism that would bridge the two, a partial materialisation through a `View`, is exercised nowhere in this sweep. Second, the failure of the eager twin — the comparison this document rests on — is not recorded in that run's trace. Its `job.finish` says `ok: true` with `failed_ranks: []`, its `metrics.json` says `degraded: false`, and the error class comes from the results file; the trace corroborates only by absence, through 18 events instead of 34 and empty fabric tables. The inference that the immediate precondition fired, rather than the credit loop timing out, rests on that run's 0.04 s span against a 12 s stall timeout and on the absence of `transport.credit_stall`. Third, token counts are estimates (`tokenizer_exact: false`): 10,779 is $\lceil 40960/3.8 \rfloor$ from a character heuristic on one repeated word, and a real tokeniser would price it near 8,192 — still 2.0× the credit, so the contrast survives, but every deferred figure here scales by about 0.76. Fourth, both budgets are benchmark parameters rather than defaults (`unexpected_limit` 4,096 against a shipped 8× eager limit; `context_budget` 32,768 against 128,000), so this pair locates a ceiling the benchmark chose. Finally, the executor is `none` with zero agent calls and the wall time is one sample dominated by a 40 KiB blob write and eight SQLite-backed rank start-ups, so nothing here is a statement about agents or about throughput.